

Celery Validation Guide

Source: docs/celery-validation.md

Celery Validation Guide

Status: internal technical documentation Last updated: 2026-05-23 Extended reference: DEVELOPER_GUIDE.md, DEPLOYMENT_GUIDE.md

Purpose

This document describes how to prove that the asynchronous pipeline based on Celery and Redis works end-to-end.

The proof should show that:

- FastAPI accepts a request and returns a `task_id`
- Celery receives the task
- the worker processes it through intermediate states
- `/status/{task_id}` eventually returns `SUCCESS`
- the final report payload is stored and returned correctly

What to run

Start the infrastructure first:

```
[powershell]
docker compose up -d redis
docker compose up -d celery-worker
docker compose --profile monitoring up -d flower
```

Start the API server from the repository root:

```
[powershell]
.\.venv\Scripts\Activate.ps1
$env:PYTHONPATH="."
python -m uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

Proof script

Run the E2E proof script from the repository root:

```
[powershell]
.\.venv\Scripts\Activate.ps1
python backend/test_e2e.py
```

The script sends a request to `/report/generate`, polls `/status/{task_id}`, and exits successfully only when Celery returns `SUCCESS`.

Expected output

A successful run should show:

- login success
- `task_id` returned from `/report/generate`
- status transitions such as `PROGRESS`
- final `SUCCESS`
- a report payload containing numeric indicators and ESG actions

Example terminal flow:

```
[text]
[KROK 1] Uruchamianie taska /report/generate dla scope 'Environmental'...
SUKCES: Task raportu w kolejce: <task_id>
<- status=PROGRESS, progress=30, etap=Wyszukiwanie kontekstu
<- status=PROGRESS, progress=80, etap=Generowanie raportu przez AI
<- status=SUCCESS, progress=100, etap=Gotowe
SUKCES: Task Celery zakonczony poprawnie i zwrocil pelny raport ESG.
```

What to show as evidence

Use these three artifacts as proof:

- Flower worker list showing the worker is Online
- Terminal output from `python backend/test_e2e.py`
- Task details in Flower showing the task moved from PROGRESS to SUCCESS

What to capture in Flower

For a Confluence screenshot, open Flower at `http://localhost:5555` and show one of these views:

Best screenshot: Worker list

Capture the worker table where you can clearly see:

- the worker name
- status Online
- non-zero Succeeded after running the proof script
- optional Active or Retried values if the task is currently running

This is the cleanest screenshot for showing that the worker is alive.

Better proof: Task details

Open the task list and click the task id created by `backend/test_e2e.py`. On the task details screen, show:

- task state changing from PROGRESS to SUCCESS
- timestamps / runtime
- the final result payload

If the task is already finished, the SUCCESS view is enough, but a live PROGRESS screen is stronger evidence.

How to make the screenshot convincing

- Keep Flower open beside the terminal where the E2E script is running.
- Start the script, wait until the task is in PROGRESS, then take a screenshot.
- After it reaches SUCCESS, take a second screenshot of the final state.
- If you can only take one screenshot, prefer the task detail page with SUCCESS and the result payload visible.

Notes

- On Windows, do not rely on the prefork Celery pool locally; use the Docker worker or `--pool=solo`.
- If `/status/{task_id}` returns FAILURE, check the worker logs in `docker compose logs -f celery-worker`.
- If the script reports duplicate upload errors, run the report-generation proof path instead, because it is stable and does not depend on a fresh PDF upload.